

Universidad Tecnológica Metropolitana

Facultad de Ingeniería / Ingeniería Civil en Electrónica
Departamento de Electricidad



INFORME DE LABORATORIO N°3 Y N°4

Sistema de visión artificial con Raspberry Pi y ESP32-CAM

Asignatura: Microcontroladores ELEB8092

Profesor: Dr. Exequiel Álvarez Olate

Experiencias: Vigilancia IoT distribuida y preparación de Raspberry Pi 4

Integrantes:

Sebastian Espinosa

Axel Valdes

Fecha: 3 de julio de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Marco teórico resumido	2
4. Materiales y software	3
5. Desarrollo	3
5.1. Arquitectura general	3
5.2. ESP32-CAM y transmisión MJPEG	4
5.3. Procesamiento en Raspberry Pi	4
5.4. Conteo de aforo	4
5.5. Preparación de Raspberry Pi para el Laboratorio 4	5
5.6. Reconocimiento de patentes	5
6. Resultados y análisis	6
6.1. Cumplimiento de las guías	7
7. Conclusiones	7
8. Entregables	7
Referencias	9

1. Introducción

Los laboratorios 3 y 4 tienen un eje común: usar la Raspberry Pi 4 como plataforma de visión artificial y delegar o concentrar tareas de captura según el hardware disponible. En el Laboratorio 3 se implementó una arquitectura distribuida donde una ESP32-CAM transmite video por Wi-Fi y la Raspberry realiza la inferencia. En el Laboratorio 4 se verificó la preparación de la Raspberry como nodo local de visión, incluyendo cámara conectada, OpenCV, YOLOv8 y OCR para patentes.

Para evitar repetir procedimientos equivalentes, este informe integra ambas experiencias en un solo flujo: primero se describe la plataforma común, luego la transmisión IoT con ESP32-CAM y finalmente la prueba ANPR/ALPR solicitada en el Laboratorio 4.

2. Objetivos

- Configurar la ESP32-CAM AI-Thinker como nodo de captura MJPEG sobre HTTP.
- Preparar la Raspberry Pi 4 para recibir video, procesar imágenes y ejecutar inferencia local.
- Implementar detección de personas/vehículos con YOLOv8 Nano en formato ONNX.
- Implementar el conteo de aforo por línea virtual solicitado como reto del Laboratorio 3.
- Validar un flujo de reconocimiento de patentes para el Laboratorio 4 y generar evidencia fotográfica.

3. Marco teórico resumido

El sistema se basa en una separación de responsabilidades. La ESP32-CAM trabaja como nodo de borde encargado de capturar y transmitir imágenes; la Raspberry Pi 4 actúa como servidor de niebla o nodo de procesamiento. Esta división es necesaria porque la ESP32-CAM no cuenta con recursos suficientes para ejecutar modelos de detección como YOLOv8 en tiempo real.

El video se transmite como MJPEG sobre HTTP, protocolo simple en el que cada frame viaja como una imagen JPEG independiente. OpenCV puede abrir este stream como si fuera una cámara local. Para la inferencia se usó YOLOv8 Nano exportado a ONNX, ejecutado mediante OpenCV DNN, evitando el uso directo de PyTorch debido a incompatibilidades observadas en la Raspberry.

Para el reconocimiento de patentes se evaluó EasyOCR, pero durante la prueba real presentó fallos de *illegal instruction* asociados a PyTorch. Por esta razón se utilizó Tesseract 5.5.0 como ruta OCR operativa, aplicando recorte de la zona de patente, escala de grises, binarización y normalización del texto.

4. Materiales y software

Tabla 1: Elementos utilizados.

Elemento	Uso
Raspberry Pi 4	Servidor de procesamiento e inferencia.
ESP32-CAM AI-Thinker	Captura y transmisión MJPEG del Laboratorio 3.
Webcam USB Logitech C170	Cámara local usada para las pruebas del Laboratorio 4.
Fuente externa 5 V	Alimentación estable para la ESP32-CAM.
Arduino CLI y core ESP32	Compilación y carga del firmware.
Python 3.13, OpenCV, NumPy	Captura, procesamiento y visualización.
YOLOv8n ONNX	Detección de personas y vehículos.
Tesseract 5.5.0	OCR funcional para patentes.

5. Desarrollo

5.1. Arquitectura general

La Figura 1 resume la arquitectura implementada. La ESP32-CAM transmite frames JPEG por Wi-Fi hacia la Raspberry Pi, donde se ejecuta el pipeline de visión artificial. Para la parte local del Laboratorio 4, la Raspberry también puede capturar directamente desde una cámara USB mediante `cv2.VideoCapture(0)`.

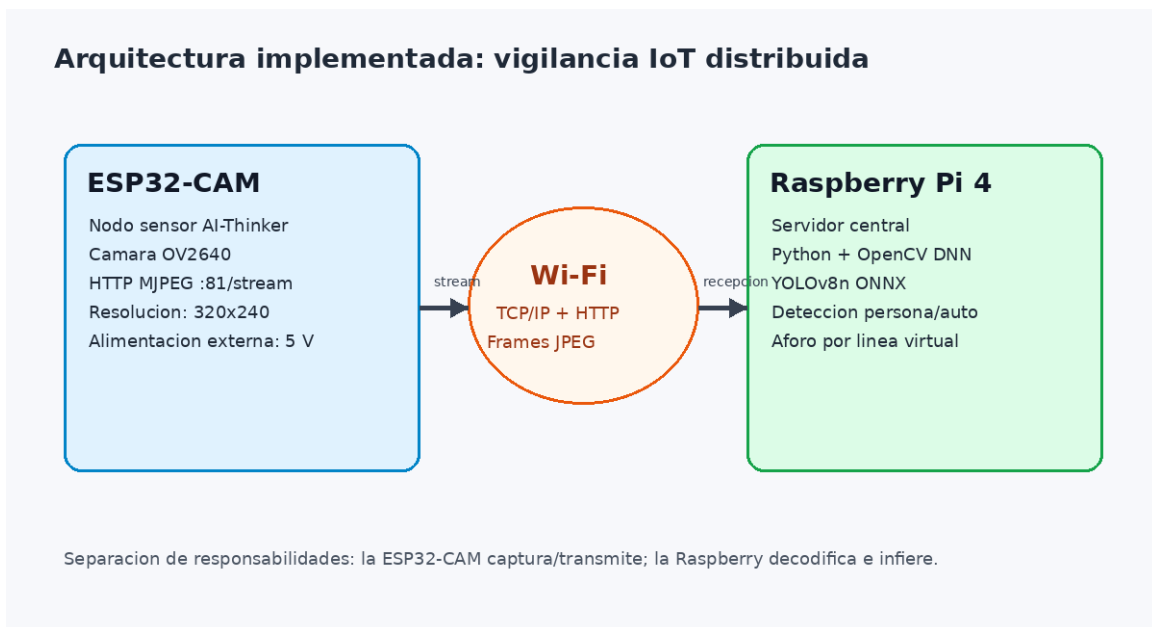


Figura 1: Arquitectura general del sistema de visión artificial.

5.2. ESP32-CAM y transmisión MJPEG

El firmware de la ESP32-CAM inicializa el sensor OV2640, configura salida JPEG y expone dos rutas HTTP: `/capture` para capturas individuales y `:81/stream` para video MJPEG continuo. Para mejorar estabilidad y FPS se redujo la resolución a QVGA (320x240), se ajustó la compresión JPEG y se desactivó el ahorro de energía Wi-Fi.

Listing 1: Configuración aplicada en la ESP32-CAM.

```
1 config.frame_size = FRAMESIZE_QVGA;
2 config.jpeg_quality = 16;
3 config.fb_count = 3;
4 config.fb_location = CAMERA_FB_IN_PSRAM;
5
6 WiFi.mode(WIFI_STA);
7 WiFi.setSleep(false);
```

5.3. Procesamiento en Raspberry Pi

La Raspberry recibe el stream desde `http://172.20.10.12:81/stream`. Para evitar latencia acumulada se implementó lectura en un hilo separado: siempre se procesa el frame más reciente. Además, YOLO se ejecuta cada cuatro frames con `--infer-every 4`, lo que mejora la fluidez de visualización en CPU.

5.4. Conteo de aforo

Para el reto del Laboratorio 3 se dibujó una línea virtual horizontal y se calcularon centroides de las personas detectadas. Cuando un centroide cruza la línea, se actualiza el contador de aforo. La Figura 2 muestra una captura real desde la ESP32-CAM con la línea de conteo.



Figura 2: Captura real desde ESP32-CAM con línea virtual de aforo.

5.5. Preparación de Raspberry Pi para el Laboratorio 4

La guía del Laboratorio 4 pide comprobar sistema operativo de 64 bits, entorno virtual, cámara y librerías de visión artificial. En esta Raspberry se verificó OpenCV, Ultralytics, EasyOCR y Tesseract. La cámara CSI/NoIR indicada por la guía no fue detectada por `rpikam`, pero sí se detectó y probó una webcam USB Logitech C170 en `/dev/video0`. La captura local funcionó a 640x480.

Listing 2: Verificación resumida del entorno.

```
1 Version de Python: 3.13.5
2 Version de OpenCV: 5.0.0
3 Ultralytics (YOLOv8): OK
4 EasyOCR: OK
5 Tesseract: 5.5.0
6 Camara funcional. Resolucion del frame: 640 x 480
```

5.6. Reconocimiento de patentes

Para la prueba ANPR/ALPR se mostró una patente desde un teléfono frente a la cámara. El texto visible fue BB-BB-10. El programa recortó la zona de la patente, aplicó binarización y ejecutó Tesseract. La lectura normalizada fue BBBB10, incluida en la lista de patentes autorizadas. La Figura 3 muestra la evidencia final.



Figura 3: Reconocimiento de patente con acceso autorizado.

6. Resultados y análisis

Tabla 2: Resultados principales.

Parámetro	Resultado
Resolución ESP32-CAM	320x240 pixeles
IP usada por ESP32-CAM	172.20.10.12
Stream	http://172.20.10.12:81/stream
Modelo de detección	YOLOv8n ONNX
Frecuencia de inferencia	1 inferencia cada 4 frames
FPS observado en prueba	2.67 FPS de visualización
OCR operativo	Tesseract 5.5.0
Patente leída	BBBB10
Resultado de acceso	Autorizado
Estado alimentación Raspberry	throttled=0x0

El rendimiento quedó limitado principalmente por CPU y por la transmisión MJPEG en Wi-Fi. La alimentación externa de 5 V para la ESP32-CAM mejoró estabilidad, evitando reinicios o cortes de stream durante las pruebas. En el caso del OCR, Tesseract resultó más estable que EasyOCR en esta instalación porque no depende de inferencia PyTorch pesada.

6.1. Cumplimiento de las guías

Tabla 3: Comparación con los puntos solicitados.

Punto solicitado	Estado	Evidencia
ESP32-CAM programada y transmitiendo por Wi-Fi	Cumplido	Firmware, stream MJPEG y configuración HTTP.
Servidor Raspberry que consume el stream	Cumplido	Script <code>rasberryesp32.py</code> con OpenCV.
Detección con YOLOv8	Cumplido	Modelo YOLOv8n exportado a ONNX.
Reto Lab 3	Cumplido	Se implementó el Reto B: conteo de aforo por línea virtual.
Entorno Python bajo PEP 668	Cumplido	Uso de entorno virtual y dependencias aisladas.
Verificación de cámara en Raspberry	Cumplido con ajuste	No se detectó cámara CSI/NoIR; se usó webcam USB funcional.
Reconocimiento de patentes	Cumplido	Lectura BBBB10 y decisión de acceso autorizado.
Alerta o salida de acceso	Cumplido	Señal visual en pantalla: acceso autorizado/no autorizado.
Repositorio de código	Parcial	Se entrega código limpio y <code>requirements.txt</code> ; falta subirlo a GitHub si el docente lo exige.
Video demostrativo	Parcial	La demostración fue en vivo; se incluye evidencia fotográfica en el informe.

7. Conclusiones

Se integraron los Laboratorios 3 y 4 en una misma plataforma de visión artificial. El Laboratorio 3 quedó cubierto mediante una ESP32-CAM transmitiendo video a la Raspberry Pi, donde se ejecutó detección con YOLOv8n ONNX y conteo de aforo por línea virtual. El Laboratorio 4 se incorporó como preparación y validación de la Raspberry para visión local, incluyendo cámara USB, entorno Python, OpenCV, OCR y prueba real de patente.

La principal limitación fue el rendimiento de inferencia en CPU. Aun así, el sistema funcionó de forma estable con resolución reducida, lectura asíncrona de frames y OCR basado en Tesseract. La evidencia final demuestra captura, procesamiento, lectura de patente y decisión de acceso autorizado.

8. Entregables

- Firmware ESP32-CAM: `esp32_cam_stream.ino`.

- Servidor Raspberry: `rasberryesp32.py`.
- Scripts Lab 4: pruebas de entorno, captura local, detección, ANPR por consola y visor web.
- Dependencias Python: `requirements.txt`.
- Modelo: `yolov8n.onnx`.
- Evidencia fotográfica incorporada en este informe.

Referencias

- [1] Ultralytics, “Yolov8 documentation,” 2024. Disponible en: <https://docs.ultralytics.com/>.
- [2] OpenCV, “Opencv documentation,” 2024. Disponible en: <https://docs.opencv.org/>.
- [3] E. Systems, “Esp32 arduino core documentation,” 2024. Disponible en: <https://docs.espressif.com/>.
- [4] E. Álvarez Olate, “Laboratorio de microcontroladores n°3: Sistema iot de vigilancia y reconocimiento distribuido,” 2026. Universidad Tecnológica Metropolitana, Departamento de Electricidad.
- [5] E. Álvarez Olate, “Laboratorio de microcontroladores n°4: Preparación de la raspberry pi4 para visión artificial,” 2026. Universidad Tecnológica Metropolitana, Departamento de Electricidad.